File Handling & Collections Framework

Unit- V

Topics to be Covered...

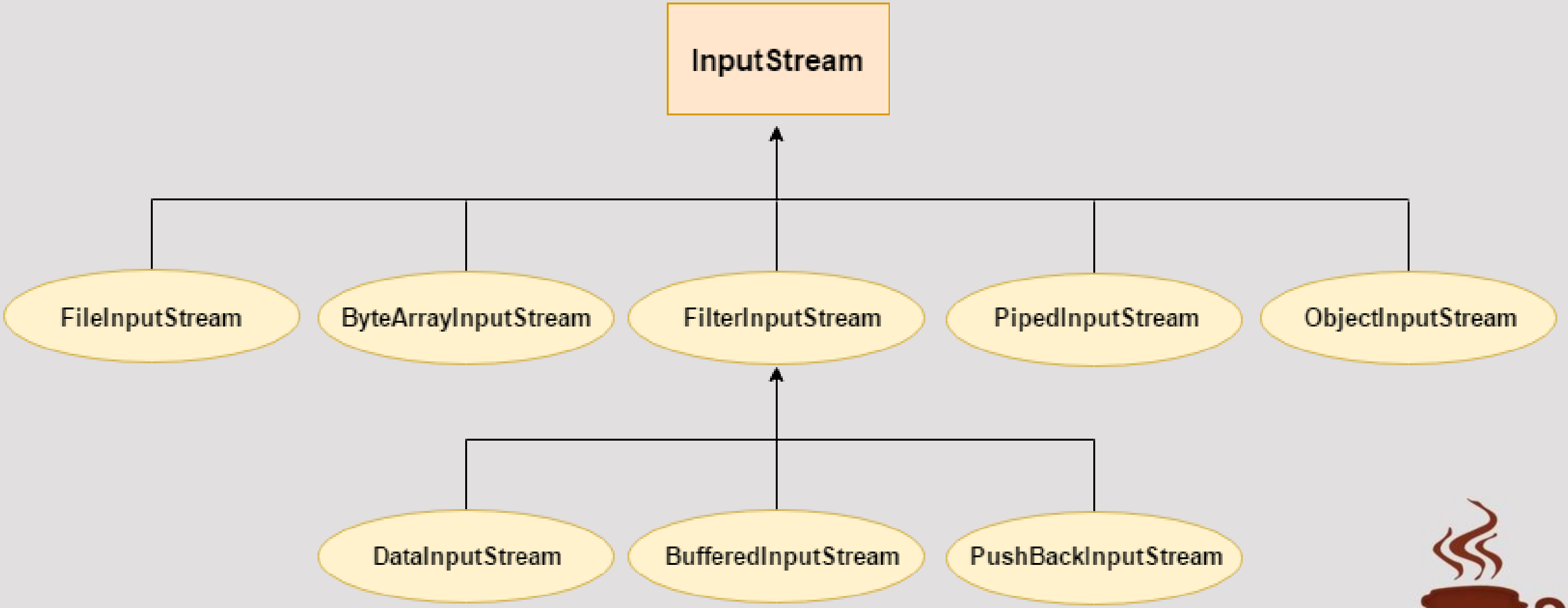
- Stream Classes
- Stream Classes Hierarchy
- Useful I/O Classes
- Creation of text file,
- Reading and Writing text files
- Generics
- Collections Framework
- Collections Class
- Map Class
- Object Class and Overriding its Methods

CO5: Develop an object-oriented program by using the files and collection framework.

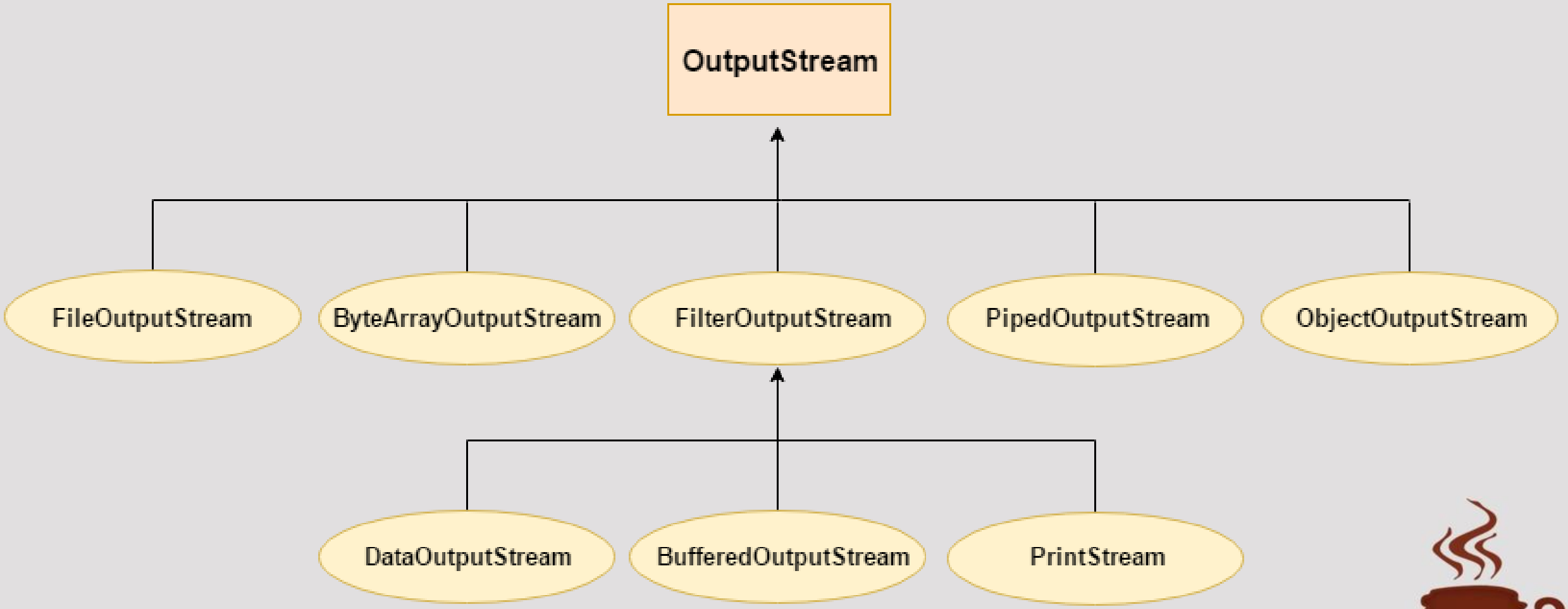
Stream Classes

- Streams are pipelines for sending and receiving information from/to Java programs.
- A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.
- In Java, 3 streams are created for us automatically. All these streams are attached with the console.
  - **1) System.out:** standard output stream: *System.out.println("simple message");*
  - **2) System.in:** standard input stream
  - **3) System.err:** standard error stream: *System.err.println("error message");*
- **InputStream**
  - Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.
- **OutputStream**
  - Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.

Stream Class Hierarchy: InputStream

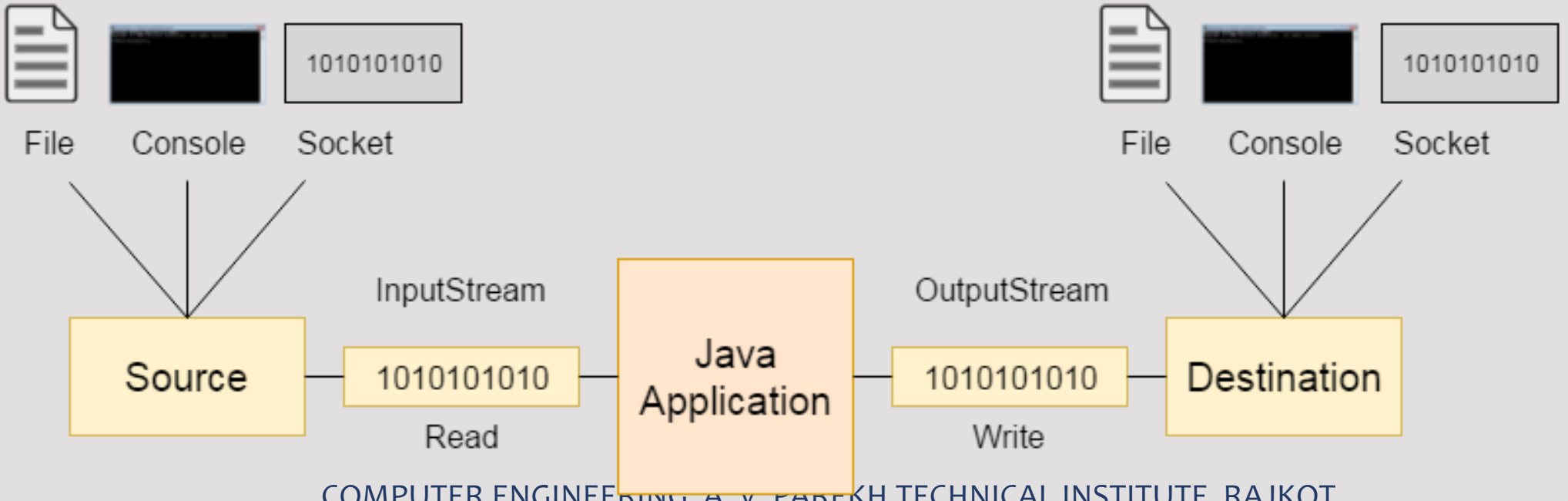


Stream Class Hierarchy: OutputStream



Useful I/O Class: FileInputStream

- In Java, FileInputStream is a class that allows you to read bytes from a file. It's a subclass of InputStream, which is used for reading bytes from various sources.
- It's part of the java.io package and is used for low-level file handling operations.
- An InputStream is automatically opened when you create object of InputStream class.
- You can explicitly close a stream with the close( ) method, or let it be closed implicitly when the object is found as a garbage.

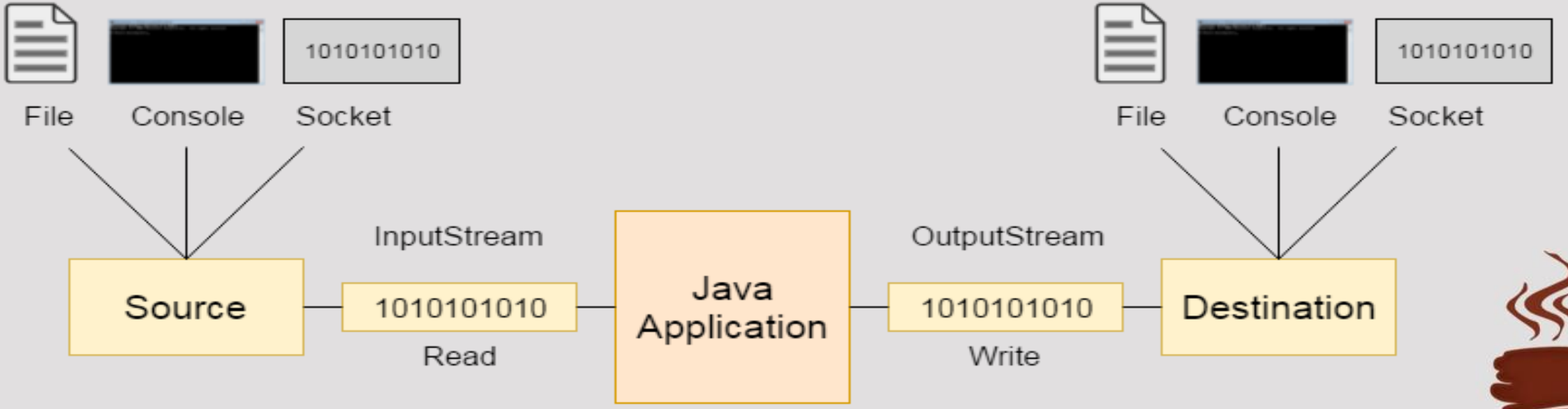


Useful I/O Class: FileInputStream

Method	Description
public abstract int read()throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
public int available()throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
public void close()throws IOException	is used to close the current input stream.

Useful I/O Class: FileOutputStream

- In Java, FileOutputStream is a class that allows you to write bytes to a file. It's a subclass of OutputStream, which is used for writing bytes to various destinations.
- Like an InputStream, an OutputStream is automatically opened when you create an object of OutputStream class.
- You can explicitly close an OutputStream with the close( ) method, or let it be closed implicitly when the object is garbage collected.



Useful I/O Class: FileOutputStream

Method	Description
public void write(int)throws IOException	is used to write a byte to the current output stream.
public void write(byte[])throws IOException	is used to write an array of byte to the current output stream.
public void flush()throws IOException	flushes the current output stream.
public void close()throws IOException	is used to close the current output stream.

Java File Handling: Two ways

- File handling classes that reads/writes java files in form of set of bytes. Those are known as **byte stream classes.**
- These classes are generally used if the file to be read/written is a binary file.
  - Ex:
    - FileInputStream
    - FileOutputStream
- File handling classes that reads/writes java files in form of set of characters. Those are known as **character stream classes.**
- These classes are generally used if the file to be read/written is a textual file.
  - Ex:
    - FileReader
    - FileWriter

Java File Handling

- **java.io** package contains various classes used to perform file handling in java.
- The **File** class from the java.io package, allows us to work with files.
- To use the File class, create an object of the class, and specify the filename or directory name:

```
import java.io.File;  // Import the File class

File myObj = new File("filename.txt"); // Specify the filename
```

Java File Handling

Method	Use
`File createNewFile()`	Creates a new, empty file. Returns true if the file was successfully created, false otherwise.
`boolean delete()`	Deletes the file or directory. Returns true if the file was successfully deleted, false otherwise.
`boolean mkdir()`	Creates a directory. Returns true if the directory was successfully created, false otherwise.
`boolean exists()`	Checks whether the file or directory exists.
`boolean isFile()`	Checks whether the File object represents a file.
`boolean isDirectory()`	Checks whether the File object represents a directory.
`boolean getName()`	Returns the name of the file or directory.
`String getPath()`	Returns the path of the file or directory.
`int length()`	Returns the length of the file in bytes.
`boolean canRead()`	Checks whether the file is readable.
`boolean canWrite()`	Checks whether the file is writable.

Creation of a new Text File

- The createNewFile() method is used to create a file in Java.
- It return true if the file is created successful else it will return false (if the file already exists or the operation failed).

Creation of a new Text File

```
import java.io.*;
public class FileCreation {
	public static void main(String[] args) {
		try {
			File file = new File("D:\\JAVA\\AOOP\\newfile.txt");
			if (!file.exists()) {
				file.createNewFile();
				System.out.println("File created Successfully.");
			}
		} catch (IOException e) {
				System.out.println("I/O Exception occurred.");
		}
	}
}
```

Output

```
File created Successfully.
<A File will be created at D:\JAVA\AOOP\newfile.txt>
```

Writing into a Text File: Using Byte Stream

```
import java.io.*;
public class FileWrite {
	public static void main(String[] args) {
		try {
			FileOutputStream fout = new
				FileOutputStream("D:\\JAVA\\AOOP\\newfile.txt");
			String s = "Welcome to AVPTI!!!";
			byte b[] = s.getBytes();
			fout.write(b);
			fout.close();
			System.out.println("Written Successfully!");
		} catch(IOException e) {
			System.out.println(e);
		}
}	}
```

Output

```
Written Successfully!
```

Reading a Text File: Using Byte Stream

```
import java.io.*;
public class FileRead {
      public static void main(String[] args) {
            try {
                  FileInputStream fin = new
                        FileInputStream("D:\\JAVA\\AOOP\\newfile.txt");
                  int i;
                  while((i = fin.read())!=-1)
                        System.out.print((char)i);
                  fin.close();
            } catch(IOException e) {
                  System.out.println(e);
            }
}     }
```

Output

```
Welcome to AVPTI!!!
```

Creating & Writing in a File: Using Character Stream

```
import java.io.FileWriter;
public class FileWriterExample {
    public static void main(String args[]){
        try{
          FileWriter fw  =
                new FileWriter("D:\\JAVA\\AOOP\\newfile1.txt");
          fw.write("Welcome to AVPTI!!!");
          fw.close();
        } catch(Exception e) { System.out.println(e); }
        System.out.println("Success...");
    }
}
```

OUTPUT

```
Success...
<A File will be created at D:\JAVA\AOOP\newfile1.txt >
```

Reading from a Text File: Using Character Stream

```
import java.io.FileReader;
public class FileReaderExample {
    public static void main(String args[]) throws Exception{
        FileReader fr = new
            FileReader("D:\\JAVA\\AOOP\\newfile1.txt");
        int i;
        while((i=fr.read())!=-1)
            System.out.print((char)i);
        fr.close();
    }
}
```

OUTPUT

```
Welcome to AVPTI!!!
```

Reading from a Text File: Using Character Stream

```
import java.io.File;
import java.util.Scanner;
public class FileReaderExample2 {
    public static void main(String args[]) throws Exception{
        File f = new
            File("D:\\JAVA\\AOOP\\newfile1.txt");
        Scanner sc = new Scanner(f);
        while(sc.hasNextLine())
            System.out.println(sc.nextLine());
    }
}
```

OUTPUT

```
Welcome to AVPTI!!!
```

Note: There is also another way to read from File using

```
FileReader fr = new FileReader("D:\\JAVA\\AOOP\\newfile1.txt")
BufferedReader fbr = new BufferedReader(fr));
```

Then using `fbr.readLine();` to read Line directly from file.

Generics

- Generics means parameterized types.
- Using Generics, it is possible to create classes that work with different data types.
- An entity such as a class, interface, or method that operates on a parameterized type is a generic entity.
- Why needed?
  - The Object is the superclass of all other classes, and Object reference can refer to any object.
  - But it does not provide type safety, meaning it can contain any type.
  - Hence Generics. (to provide type safety)

Generics: Example

```
class Test<T> {
    // An object of type T is declared
    T obj;
    Test(T obj) { this.obj = obj; } // constructor
    public T getObject() { return this.obj; }
}
```

Generics: Example

```
class Main {
    public static void main(String[] args) {
        // instance of Integer type
        Test<Integer> iObj = new Test<Integer>(15);
        System.out.println(iObj.getObject());

        // instance of String type
        Test<String> sObj
            = new Test<String>("AVPTI");
        System.out.println(sObj.getObject());
}   }
```

OUTPUT

```
15
AVPTI
```


Generics

- We can also pass multiple Type parameters in Generic classes.

```
class Test<T, U>
```

- We can Instantiate this Class as:

```
Test <String, Integer> obj  = new
    Test<String, Integer>("AVPTI", 15);
```

- Like wise generics can also be applied to methods, if not required in the whole class.

```
static <T> void genericDisplay(T element)
```

- The above T type will be limited to method genericDisplay().
- Generics can only work with reference types(Objects/Array Only)

Generics

- Advantages of using Generics:
  - **Code Reusability:** You can write a method, class, or interface once and use it with any type.
  - **Type Safety:** Generics ensure that errors are detected at compile time rather than runtime, promoting safer code.
  - **No Need for Type Casting:** The compiler automatically handles casting, removing the need for explicit type casting when retrieving data.
  - **Code Readability and Maintenance:** By specifying types, code becomes easier to read and maintain.
  - **Generic Algorithms:** Generics allow for the implementation of algorithms that work across various types, promoting efficient coding practices.

Collections Framework

- Collections Framework is a set of classes and interfaces that provide commonly reusable collection data structures, these data structures allow you to store, manipulate, and access groups of objects efficiently.
- In Java, a separate framework named the “Collection Framework” has been defined in JDK 1.2 which holds all the collection classes and interface in it.
- The Collection interface (java.util.Collection) and Map interface (java.util.Map) are the two main “root” interfaces of Java collection classes.
- Before the Collection Framework was introduced, the standard methods for grouping Java objects (or collections) were Arrays or Vectors, or HashTables.

ArrayList<E>

- ArrayList is a class in the Collections Framework that implements the List interface, It provides a dynamic array-like structure that can grow or shrink in size as needed (Unlike simple array which has fixed sizes decided at declaration time), ArrayList<E> is part of the java.util package.
- Advantages:
  - Dynamic Sizing
  - Ordered Collection
  - Random Access
  - Improved Performance
- Syntax: (Object Creation)

```
import java.util.ArrayList;   // import the ArrayList class
ArrayList<String> cars = new ArrayList<String>( );
// Create an ArrayList object
```

ArrayList: Add Item (add() method)

- Signature: `public booelan add(E element)`

```
import java.util.ArrayList;
public class AlistAdd {
      public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            System.out.println(cars);
}     }
```

OUTPUT

```
[Volvo, BMW, Ford, Mazda]
```

ArrayList: Add Item (add() method)

- Signature: `public void add(int i, E element)`

```
import java.util.ArrayList;
public class AlistAdd {
      public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add(1, "Mazda");
            System.out.println(cars);
}     }
```

OUTPUT

```
[Volvo, Mazda, BMW, Ford]
```

ArrayList: Access Item (get() method)

- Signature: `public E get(int index)`

```
import java.util.ArrayList;
public class AlistAccess {
      public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            System.out.println(cars.get(1));
}     }
```

OUTPUT

```
BMW
```

ArrayList: Update Item (set() method)

- Sugnature: `public E set(int index, E newItem)`

```
import java.util.ArrayList;
public class AlistSet {
      public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            cars.set(2, "Audi");
            System.out.println(cars);
}     }
```

OUTPUT

```
[Volvo, BMW, Audi, Mazda]
```

ArrayList: Remove an Item (remove() method)

- Signature: `public E remove(int index)`

```
import java.util.ArrayList;
public class AlistRemove {
      public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            String elment = cars.remove(2);
            System.out.println(element);
}     }
```

OUTPUT

```
Ford
```

ArrayList: Remove all Items (clear() method)

- Signature: public void clear()

```
import java.util.ArrayList;
public class AlistClear {
      public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            cars.clear();
            System.out.println(cars);
}     }
```

OUTPUT

```
[ ]
```


ArrayList: Get the size (size() method)

- Signature: public int size()

```
import java.util.ArrayList;
public class AlistSize {
      public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            System.out.println(cars.size());
}    }
```

OUTPUT

4

ArrayList: Iteration (forEach() method)

- Signature: `public void forEach(Consumer<? super E> action)`

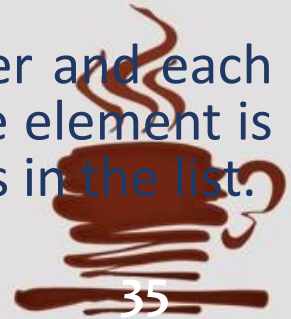
```
import java.util.ArrayList;
public class AlistForEach {
      public static void main(String[] args) {
            ArrayList<String> cars =
                  new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            cars.forEach((car) -> System.out.println(car));
}     }
```

OUTPUT

```
Volvo
BMW
Ford
Mazda
```

LinkedList<E>

- Doubly-linked list implementation of the List and Deque interfaces.
- Part of java.util Package.
- The LinkedList class has all of the same methods as the ArrayList class, this means that you can add items, change items, remove items and clear the list in the same way.
- Just the internal storage method is different for LinkedList.
- ArrayList working
  - The ArrayList class has a regular array inside it, When an element is added, it is placed into the array. If the array is not big enough, a new, larger array is created to replace the old one and the old one is removed.
- LinkedList working
  - The LinkedList stores its items in "containers".The list has a link to the first container and each container has a link to the next container in the list. To add an element to the list, the element is placed into a new container and that container is linked to one of the other containers in the list.



LinkedList<E>

- Syntax: (Object Creation)

```
import java.util.LinkedList;  // import the LinkedList class
LinkedList<String> cars = new LinkedList<String>( );
// Create an LinkedList object
```

LinkedList: Add Item (add() method)

- Signature: `public boolean add(E element)`

```
import java.util.LinkedList;
public class LlistAdd {
      public static void main(String[] args) {
            LinkedList<String> cars = new LinkedList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            System.out.println(cars);
}     }
```

OUTPUT

```
[Volvo, BMW, Ford, Mazda]
```

ArrayList: Add Item (add() method)

- Signature: `public void add(int i, E element)`

```
import java.util.LinkedList;
public class LlistAdd {
      public static void main(String[] args) {
            LinkedList<String> cars = new LinkedList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add(1, "Mazda");
            System.out.println(cars);
}    }
```

OUTPUT

```
[Volvo, Mazda, BMW, Ford]
```

LinkedList: Some Add & Remove Methods

Method	Use
`void addFirst(E e)`	Adds an item to the beginning of the list.
`void addLast(E e)`	Add an item to the end of the list
`E removeFirst()`	Remove an item from the beginning of the list.
`E removeLast()`	Remove an item from the end of the list
`void push(E e)`	Inserts an element onto a stack represented by the list.
`E pop()`	Pops an element onto a stack represented by the list
`E peek()`	Returns the top element of the stack represented by the list (does not remove).
`E peekFirst()`	Returns the head element of the stack represented by the list (does not remove).
`E peekLast()`	Returns the last element of the stack represented by the list (does not remove).
`int size()`	Returns the size of the LinkedList
`Object[] toArray()`	Returns an Equivalent array from the List

LinkedList

```
import java.util.LinkedList;
public class LinkedlistDemo {
	public static void main(String[] args)  {
		LinkedList<String> cars = new LinkedList<String>();
		cars.add("Volvo");
		cars.add("BMW");
		cars.add("Ford");
		cars.add("Mazda");
		System.out.println(cars);
		cars.addFirst("Audi");
		System.out.println(cars);
		cars.addLast("Lexux");
		System.out.println(cars);
		cars.removeFirst();
		System.out.println(cars);
		cars.removeLast();
		System.out.println(cars);
	}
}
```

OUTPUT

```
[Volvo, BMW, Ford, Mazda]
[Audi, Volvo, BMW, Ford, Mazda]
[Audi, Volvo, BMW, Ford, Mazda, Lexus]
[Volvo, BMW, Ford, Mazda, Lexus]
[Volvo, BMW, Ford, Mazda]
```

HashSet<E>

- This class implements the Set interface, backed by a hash table (actually a HashMap instance).
- Part of java.util Package.
- Stores only unique values with null values allowed, does not allow duplicates.
- Unordered Set.
- Not Thread-safe class.
- Syntax: (Object Creation)

```
import java.util.HashSet;     // import the HashList class
HashSet<String> cars = new HashSet<String>( );
// Create an HashSet object
```

HashSet: Add an Item(add() method)

- Signature: `public boolean add(E e)`

```
import java.util.HashSet;
public class HSetAdd {
      public static void main(String[] args) {
            HashSet<String> cars = new HashSet<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            System.out.println(cars);
}     }
```

OUTPUT

```
[Volvo, Mazda, Ford, BMW]
```

HashSet: Check for an Item(contains() method)

- Signature: `public boolean contains(Object o)`

```
import java.util.HashSet;
public class HSetContains {
      public static void main(String[] args) {
            HashSet<String> cars = new HashSet<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
            System.out.println(cars.contains("Ford")));
}    }
```

OUTPUT

```
true
```

HashSet: Remove an Item(remove() method)

- Signature: `public boolean remove(Object o)`

```
import java.util.HashSet;
public class HSetRemove {
      public static void main(String[] args) {
            HashSet<String> cars = new HashSet<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            System.out.println(cars.remove("Ford")));
            System.out.println(cars.remove("Mazda")));
}    }
```

OUTPUT

```
true
false
```

HashSet: Remove all Items(clear() method)

- Signature: `public boolean remove(Object o)`

```
import java.util.HashSet;
public class HSetRemoveAll {
      public static void main(String[] args) {
            HashSet<String> cars = new HashSet<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.clear();
            System.out.println(cars);
}    }
```

OUTPUT

```
[ ]
```

HashSet: Check emptiness (isEmpty() method)

- Signature: `public boolean isEmpty()`

```
import java.util.HashSet;
public class HSetEmpty {
      public static void main(String[] args) {
            HashSet<String> cars = new HashSet<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            System.out.println(cars.isEmpty());
}   }
```

OUTPUT

```
false
```

HashSet: Get its Size (size() method)

- Signature: `public int size()`

```
import java.util.HashSet;
public class HSetSize {
      public static void main(String[] args) {
            HashSet<String> cars = new HashSet<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            System.out.println(cars.size());
}    }
```

OUTPUT

```
3
```

HashSet: Iteration using ForEach Loop

```
import java.util.HashSet;
public class HSetIteration {
     public static void main(String[] args)  {
          HashSet<String> cars = new HashSet<String>();
          cars.add("Volvo");
          cars.add("BMW");
          cars.add("Ford");
          cars.add("BMW");
          cars.add("Mazda");
          for (String i : cars)
                    System.out.println(i);
  } }
```

OUTPUT

```
Volvo
Mazda
Ford
BMW
```

Note: Likewise we can iterate with all the Collection Framework Objects like ArrayList, LinkedList with a For-Each Loop.

Map Class: HashMap<K, V>

- Hash table based implementation of the Map interface.
- Provides all Map Operations
- Allows null values as well as the null key.
- Not Thread-safe.
- HashMap allows for efficient key-based retrieval, insertion, and removal with average O(1) time complexity like, get & put
- It maps Key using its hash and the value can be accessed directly by finding the key.
- Duplicate Elements not allowed in HashMap, if you try to insert the duplicate key in HashMap, it will replace the element of the corresponding key.
- Syntax: (Object Creation)

```
import java.util.HashMap;  // import the HashMap class
HashMap<int, String> weekdays = new HashMap<int, String>( );
// Create an HashMap object
```

HashMap: Add Item (put() method)

- Signature: `public V put(K key, V value)`

```
import java.util.HashMap;
public class HMapAdd {
     public static void main(String[] args)  {
          HashMap<String, String> capitals =
               new HashMap<String, String>();
          capitals.put("England", "London");
          capitals.put("Germany", "Berlin");
          capitals.put("Norway", "Oslo");
          capitals.put("USA", "Washington DC");
          System.out.println(capitals);
  } }
```

OUTPUT

```
{USA=Washington DC, Norway=Oslo, England=London, Germany=Berlin}
```

HashMap: Access an Item(get() method)

- Signature: `public V get(Object key)`

`public V getOrDefault(Object key, V defaultValue)`

```
import java.util.HashMap;
public class HMapGet {
    public static void main(String[] args)  {
        HashMap<String, String> capitals =
            new HashMap<String, String>();
        capitals.put("England", "London");
        capitals.put("Germany", "Berlin");
        capitals.put("Norway", "Oslo");
        capitals.put("USA", "Washington DC");
        System.out.println(capitals.get("Norway"));
  } }
```

OUTPUT

```
Oslo
```

HashMap: Remove an Item(remove() method)

- Signature: `public void remove(Object key)`

```
public void remove(Object key, V value)
//Only removes if key is mapped to value
```

```
import java.util.HashMap;
public class HMapGet {
    public static void main(String[] args)  {
        HashMap<String, String> capitals =
            new HashMap<String, String>();
        capitals.put("England", "London");
        capitals.put("Germany", "Berlin");
        capitals.put("USA", "Washington DC");
        System.out.println(capitals.remove("USA"));
  } }
```

OUTPUT

```
Washington DC
```

HashMap: Remove all (clear() method)

- Signature: `public void clear()`

```
import java.util.HashMap;
public class HMapGet {
     public static void main(String[] args)  {
          HashMap<String, String> capitals =
               new HashMap<String, String>();
          capitals.put("England", "London");
          capitals.put("Germany", "Berlin");
          capitals.put("USA", "Washington DC");
          capitals.clear();
          System.out.println(capitals);
  } }
```

OUTPUT

```
{ }
```

HashMap: Replace an Item (replace() method)

- Signature: `public V replace(Object key, V value)`

```
public boolean replace(Object key, V oValue, V nValue)
//Performs replace only if key is mapped to oValue
```

```
import java.util.HashMap;
public class HMapReplace {
    public static void main(String[] args)  {
        HashMap<String, String> capitals =
            new HashMap<String, String>();
        capitals.put("England", "London");
        capitals.put("India", "Berlin");
    System.out.println(capitals.replace("India", "Delhi"));
  } }
```

OUTPUT

```
Berlin
```

HashMap: get its size (size() method)

- Signature: `public int size()`

```
import java.util.HashMap;
public class HMapSize {
     public static void main(String[] args)  {
          HashMap<String, String> capitals =
               new HashMap<String, String>();
          capitals.put("England", "London");
          capitals.put("Germany", "Berlin");
          capitals.put("USA", "Washington DC");
          System.out.println(capitals.size());
  } }
```

OUTPUT

```
3
```

HashMap: Check for emptiness (isEmpty() method)

- Signature: `public boolean isEmpty()`

```
import java.util.HashMap;
public class HMapEmpty {
     public static void main(String[] args)  {
          HashMap<String, String> capitals =
               new HashMap<String, String>();
          capitals.put("England", "London");
          capitals.put("Germany", "Berlin");
          capitals.put("USA", "Washington DC");
          capitals.clear()
          System.out.println(capitals.isEmpty());
  } }
```

OUTPUT

```
true
```

HashMap: Some other Useful Methods

Method	Use
`Set<K> keySet()`	Returns all keys in form of Set<K> Object.
`Collection<V> values()`	Returns all values as Collection<V> Object.
`void forEach()`	Used for Iteration over the HashMap. Takes a BiFunction<K, V> argument to specify the function that is needed to be performed on each key-value pair.
`booelan containsKey(Object key)`	Returns true if key is present in HashMap.
`boolean cotainsValue(Object value)`	Returns true if value is present in HashMap.
`V putIfAbsent(K key, V value)`	If key is present and the value is not associated with it, the value will be associated and returns null. If already a value is available then returns that value